

Dundu Store

Hosting Guide — step by step

Backend API on **Render** · Website and Admin panel on **Vercel** · Mobile app with **Expo EAS**

Repository: github.com/adarsh-techy/Dundu_Store

Prepared 28 September 2026. Keep this next to CEREBRUM.md in the repository.

Contents

- 1 The big picture — what runs where
- 2 Before you start — accounts and one-time preparation
- 3 Step 1 — Put the code on GitHub
- 4 Step 2 — Backend API + database on Render
- 5 Step 3 — Website on Vercel
- 6 Step 4 — Admin panel on Vercel
- 7 Step 5 — Connect the pieces (CORS, callback URLs)
- 8 Step 6 — Create the first super admin
- 9 Step 7 — Third-party keys: Cloudinary, Razorpay, Twilio, Google
- 10 Step 8 — Mobile app with Expo (developers, testers, stores)
- 11 Step 9 — Custom domains and HTTPS
- 12 Day-to-day: deploying updates, logs, environment changes
- 13 Troubleshooting
- 14 Final checklist

1. The big picture

The project has one backend and three frontends. Each piece has one home:

Part	Folder in repo	Hosted on	Plan	Public address (example)
Backend API (Express + Postgres)	backend/	Render — Web Service	Free (sleeps) or Starter	https://dundu-backend-demo.onrender.com
Postgres database	—	Render — Postgres	Free	internal only
Customer website (React/Vite)	web/	Vercel — project 1	Hobby (free)	https://dundu-web-demo.vercel.app
Admin panel (React/Vite)	admin/	Vercel — project 2	Hobby (free)	https://dundu-store.vercel.app
Mobile app (Expo / React Native)	mobile/	Expo EAS → Play Store / TestFlight	Free tier	installed on phones

How they talk to each other:

- The website, admin panel and mobile app all call the same backend API over HTTPS.
- The backend talks to Postgres (same Render account) and to third-party services: Cloudinary (images), Razorpay (payments), Twilio (WhatsApp/OTP), Google (login).
- The two Vercel apps proxy **/api** and **/uploads** to the Render backend (configured in **web/vercel.json** and **admin/vercel.json**), so they work even before you set an API URL.
- Every push to the **main** branch on GitHub redeploys Render and both Vercel projects automatically.

Note. The mobile app is not 'hosted'. It is built into an installable app by Expo EAS and distributed to phones (Expo Go for developers, APK / TestFlight for testers, Play Store / App Store for customers).

2. Before you start

Create these accounts (all free to start). Use the same email everywhere and sign in with GitHub where offered.

Service	Sign up at	Used for
GitHub	github.com	Source code; Render and Vercel deploy from it
Render	render.com	Backend API + Postgres
Vercel	vercel.com	Website + Admin panel
Cloudinary	cloudinary.com	Product / banner image storage (required on Render)
Razorpay	razorpay.com	Online payments (test mode first)
Twilio	twilio.com	WhatsApp OTP and order notifications
Google Cloud	console.cloud.google.com	Google sign-in (optional)
Expo	expo.dev	Mobile app builds (EAS)

On your Mac you need: Node.js 20 or newer, git, and (for mobile) the Expo CLI (`npm i -g eas-cli`).

Generate a strong JWT secret now and keep it somewhere safe. In Terminal:

Note. The backend refuses to start in production if **JWT_SECRET** is missing or still looks like a placeholder (for example 'your_jwt_secret'). This is intentional.

3. Step 1 — Put the code on GitHub

The repository already exists at github.com/adarsh-techy/Dundu_Store and the local folder is connected to it. To publish new changes:

What is **never** pushed (protected by `.gitignore`): `.env` files, `node_modules`, build output (`dist`), uploaded images. The `.env.example` files are pushed so others know which settings exist.

Note. If a push fails with 'rejected', run `git pull --rebase origin main` first, then push again.

4. Step 2 — Backend API + database on Render

There are two ways. **Option A (Blueprint)** creates everything with one click from the `render.yaml` file in the repo. **Option B (Manual)** is the same result done by hand. Use A.

Option A — Blueprint (recommended)

- 1 Go to **dashboard.render.com** and sign in with GitHub.
- 2 Click **New +** (top right) → **Blueprint**.
- 3 Select the repository **adarsh-techy/Dundu_Store** (click Connect if asked).
- 4 Render reads `render.yaml` and lists two resources: **dundu-db** (Postgres) and **dundu-api** (Web Service). Give the blueprint the name **dundu**.
- 5 Click **Apply**. Render creates the database, generates a random **JWT_SECRET**, wires **DATABASE_URL** automatically, installs dependencies, runs database migrations and starts the API. This takes 3 to 5 minutes.
- 6 Open the **dundu-api** service → **Logs**. Wait for the line: ✓ Dundu API server running on port 10000 (production).
- 7 Copy the service URL shown at the top (for example **https://dundu-backend-demo.onrender.com**). You will need it for Vercel and the mobile app.

Option B — Manual

Database first:

- 1 **New +** → **Postgres**. Name **dundu-db**, Region **Singapore**, Plan **Free** → Create Database.
- 2 When status is Available, open it and copy the **Internal Database URL**.

Then the web service:

- 1 **New +** → **Web Service** → pick the repository.
- 2 Fill in the fields exactly as in the table below and click **Create Web Service**.

Field	Value
Name	dundu-api
Region	Singapore (same as the database)
Branch	main
Root Directory	backend
Runtime	Node
Build Command	npm ci
Start Command	npm run migrate && npm start
Instance Type	Free (or Starter to avoid sleeping)
Health Check Path (Settings tab)	/health

Environment variables (Render → service → Environment → 'Add from .env', paste, then edit values):

Add these later when you have the keys (Step 7): CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET, RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET, TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET.

What every variable means

Variable	Meaning
NODE_ENV	Must be production. Enables strict CORS, hides error details, disables the dev OTP.
PORT	Render overrides this with its own port; leaving 5000 is fine.
TRUST_PROXY	Render sits behind a proxy; 1 makes rate limiting see the real client IP.
JWT_SECRET	Signs login tokens. Long random string. Changing it logs everyone out.
JWT_EXPIRES_IN	Token lifetime, e.g. 7d.
DATABASE_URL	Postgres connection string. Blueprint fills it; edit here if you move the database.
WEB_CLIENT_URL / ADMIN_CLIENT_URL	Exact Vercel URLs (https://, no trailing slash). Requests from other origins are refused.
GOOGLE_CALLBACK_URL	Render URL + /api/auth/google/callback. Must match the Google Cloud console.
ALLOW_DEV_OTP	Never set on Render. Only for local development (fixed OTP 123456).
RUN_MIGRATIONS_ON_START	Defaults to true: the API applies database migrations every boot. Leave unset.

Verify

Open these in a browser (replace the host with yours):

Note. Free instances sleep after 15 minutes without traffic; the first request afterwards can take 30–60 seconds. Upgrade to Starter (about \$7/month) when real customers arrive.

5. Step 3 — Website on Vercel

- 1 Go to **vercel.com** → sign in with GitHub → **Add New...** → **Project**.
- 2 Find **Dundu_Store** → **Import**.
- 3 Set the project up as in the table below.
- 4 Open **Environment Variables** and add the values shown after the table.
- 5 Click **Deploy**. After about a minute you get a URL such as **https://dundu-web-demo.vercel.app**.

Field	Value
Project Name	dundu-web
Framework Preset	Vite (detected automatically)
Root Directory	click Edit → choose web
Build Command	leave default (npm run build)
Output Directory	leave default (dist)

- **VITE_API_BASE_URL**: your Render URL, no trailing slash. (Even without it the site works, because vercel.json proxies /api to Render, but set it anyway.)
- **VITE_RAZORPAY_KEY_ID**: the public key id from Razorpay (starts with rzp_test_ or rzp_live_). Leave it out until you have one; cash on delivery still works.
- **VITE_SUPPORT_WHATSAPP**: the store's WhatsApp number with country code, digits only. Shown in the footer and help pages.

Note. Vercel environment variables are baked in at build time. After changing one, go to Deployments →  on the latest → **Redeploy**.

6. Step 4 — Admin panel on Vercel

Same as the website, second project:

- 1 **Add New...** → **Project** → import **Dundu_Store** again.
- 2 Project Name **dundu-admin**, Root Directory **admin**, framework Vite.
- 3 Environment variable: `VITE_API_BASE_URL=https://dundu-backend-demo.onrender.com`
- 4 **Deploy** → you get **https://dundu-store.vercel.app**.

Restrict who can reach the admin panel (recommended)

Vercel → dundu-admin → Settings → **Deployment Protection** → turn on **Vercel Authentication** or a password. The panel already needs an admin login, but this adds a second wall in front of it.

7. Step 5 — Connect the pieces

The backend only accepts browser requests from the URLs you list. Now that the two Vercel URLs exist:

- 1 Render → **dundu-api** → **Environment**.
- 2 Set **WEB_CLIENT_URL** to your website URL and **ADMIN_CLIENT_URL** to your admin URL — exactly as Vercel shows them, with `https://` and no trailing slash.
- 3 Click **Save, rebuild, and deploy**. Wait for the deploy to finish.

Note. Symptom when this is wrong: the site loads but every page shows 'Network error' and the browser console says 'blocked by CORS policy'.

8. Step 6 — Create the first super admin

A fresh database has no users. The admin panel has a one-time setup page for this:

- 1 Open <https://dundu-store.vercel.app/register>.
- 2 Enter your name, email and a strong password → **Register**.
- 3 Log in at **/login** with those details.

The register page locks itself as soon as one admin exists ('Admin setup already complete'). From then on, create staff accounts inside the panel: **Customers** → **Admins**. Branch admins only see the areas you grant (orders, returns, loyalty, reports, wallets); everything else is super-admin only.

Then add your catalogue: **Catalog** → **Categories** first, then **Products** (with images), then banners, coupons and announcements. Deleted items go to **System** → **Trash** and can be restored for 30 days. Every admin action is recorded in **System** → **Audit Log**.

9. Step 7 — Third-party keys

Cloudinary (images) — required on Render

Render's disk is wiped on every deploy, so product images must live in Cloudinary. Without it, uploaded images disappear after the next deploy.

- 1 Sign up at cloudinary.com (free plan is plenty).
- 2 Dashboard → copy **Cloud name**, **API Key**, **API Secret**.
- 3 Render → dundu-api → Environment → add `CLOUDINARY_CLOUD_NAME`, `CLOUDINARY_API_KEY`, `CLOUDINARY_API_SECRET` → Save, rebuild, and deploy.
- 4 The backend switches to Cloudinary automatically when all three are present. Re-upload any images added before this step.

Razorpay (payments)

- 1 Sign up at razorpay.com. Start in **Test Mode**.
- 2 Settings → API Keys → Generate. Copy **Key Id** and **Key Secret**.
- 3 Render: `RAZORPAY_KEY_ID` and `RAZORPAY_KEY_SECRET`.
- 4 Vercel (dundu-web): `VITE_RAZORPAY_KEY_ID` (Key Id only, never the secret) → Redeploy.
- 5 Test with card 4111 1111 1111 1111, any future expiry, any CVV. When ready, complete KYC, switch to Live Mode, generate live keys and replace both places.

Twilio (WhatsApp OTP and notifications)

- 1 Sign up at twilio.com → Console → copy **Account SID** and **Auth Token**.
- 2 Messaging → Try it out → **Send a WhatsApp message** → join the sandbox with your phone (send the join code shown).
- 3 Render: `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_WHATSAPP_FROM=what sapp: +14155238886` (sandbox number).
- 4 For real customers, apply for a WhatsApp Business sender in Twilio and replace `TWILIO_WHATSAPP_FROM` with your approved number.

Note. Without Twilio the backend runs in 'mock' mode: OTPs are printed in the Render logs instead of being sent. Phone login then only works for testers who can read the logs.

Google sign-in (optional)

- 1 console.cloud.google.com → create a project → APIs & Services → **Credentials** → Create credentials → **OAuth client ID** → Web application.
- 2 Authorised redirect URI:
`https://dundu-backend-demo.onrender.com/api/auth/google/callback`
- 3 Copy Client ID and Client secret → Render: `GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`; confirm `GOOGLE_CALLBACK_URL` matches the URI above and `WEB_CLIENT_URL` is the website URL (users are sent back to `WEB_CLIENT_URL/auth/callback`).

10. Step 8 — Mobile app with Expo

Three stages, from quickest to most public:

Stage	Who	How it reaches the phone	Needs Apple/Google developer account?
A. Development	Developers	Expo Go app + QR code	No
B. Testing	Testers, staff	EAS Build → APK (Android) / TestFlight (iOS)	iOS: yes (Apple \$99/yr). Android APK: no
C. Customers	Everyone	Play Store / App Store	Yes (Google \$25 once, Apple \$99/yr)

Point the app at the live API

Create `mobile/.env` (it is git-ignored) with:

A. Development with Expo Go

- 1 Install **Expo Go** from the Play Store / App Store on the phone.
- 2 On the Mac: `cd mobile && npm install && npx expo start`
- 3 Scan the QR code with the phone (Android: inside Expo Go; iPhone: Camera app). The app opens and talks to the Render API.
- 4 Changes you save on the Mac reload on the phone instantly.

B. Test builds with EAS

- 1 `npm i -g eas-cli` then `eas login` (expo.dev account).
- 2 Inside `mobile/`: `eas build:configure` (creates `eas.json`).
- 3 Android APK for testers: `eas build -p android --profile preview`. When it finishes, EAS prints a link; testers open it on the phone and install the APK.
- 4 iOS for testers: `eas build -p ios --profile preview` then `eas submit -p ios` to TestFlight (needs the Apple developer account and an App Store Connect app record).
- 5 Push JavaScript-only fixes without a new build: `eas update --branch preview --message "fix"`.

C. Store release

- 1 Set the final app name, icon, bundle identifiers and version in `mobile/app.json`.
- 2 Switch `EXPO_PUBLIC_RAZORPAY_KEY_ID` to the live key.
- 3 `eas build -p android --profile production` → `eas submit -p android` (Play Console) and the same with `-p ios` (App Store Connect).

- 4 Fill in store listings, screenshots, privacy policy URL (the website's /help/privacy page works) and submit for review.

Note. The API URL is compiled into the app. If you ever move the backend, rebuild (or eas update) the app.

11. Step 9 — Custom domains and HTTPS

Optional, but customers trust **dundu.in** more than a `vercel.app` address. Both providers give free HTTPS certificates.

Where	Domain	Steps
Vercel (website)	dundu.in and www.dundu.in	Project → Settings → Domains → Add. Vercel shows the DNS records (A 76.76.21.21 and CNAME <code>cname.vercel-dns.com</code>). Add them at your registrar.
Vercel (admin)	admin.dundu.in	Same, on the dundu-admin project.
Render (API)	api.dundu.in	Service → Settings → Custom Domains → Add → create the CNAME Render shows.

After the domains work, update every place that stores a URL:

- Render: `WEB_CLIENT_URL=https://dundu.in`,
`ADMIN_CLIENT_URL=https://admin.dundu.in`,
`GOOGLE_CALLBACK_URL=https://api.dundu.in/api/auth/google/callback`.
- Vercel (both): `VITE_API_BASE_URL=https://api.dundu.in` and redeploy; also edit the two `vercel.json` rewrite destinations in the repo.
- Google Cloud: add the new redirect URI. Mobile: `EXP0_PUBLIC_API_URL` and rebuild.

12. Day-to-day operations

Deploying an update

Commit and push to **main**. Render rebuilds the API (2–4 min) and runs any new database migrations automatically; Vercel rebuilds both apps (about 1 min). Nothing else to do.

Watching logs

- Render → dundu-api → **Logs** (live). Errors appear in red; the API prints one line per boot stage.
- Vercel → project → Deployments → click a deployment → Build Logs (build problems) and Runtime Logs (rare, the apps are static).
- Inside the admin panel: **System** → **Audit Log** shows every admin action; **Users** → **User Activity** shows customer logins.

Changing a setting

- Render env vars: Environment tab → edit → **Save, rebuild, and deploy**.
- Vercel env vars: Settings → Environment Variables → edit → **Redeploy** the latest deployment.
- Store settings (delivery charge, free-delivery threshold, COD on/off, loyalty rules, return rules) are edited inside the admin panel under **System** → **Settings**, no redeploy needed.

Database backups

Render Free Postgres has no automatic backups and expires after 90 days unless upgraded. For anything real, use the Starter database plan (daily backups) or take manual dumps:

13. Troubleshooting

Symptom	Cause	Fix
Render log: 'JWT_SECRET must be set to a real value'	Placeholder secret in production	Set a random JWT_SECRET (openssl rand -hex 48) and redeploy
Render log: 'getaddrinfo ENOTFOUND ...'	DATABASE_URL is wrong or a placeholder	Paste the Internal Database URL from the Postgres page
API returns 'relation "users" does not exist'	Migrations did not run	Migrations now run at boot; redeploy. Or set Start Command to npm run migrate && npm start
Browser: 'blocked by CORS policy'	WEB_CLIENT_URL / ADMIN_CLIENT_URL don't match the Vercel URL	Fix the values exactly (https://, no trailing slash), Save, rebuild, and deploy
Login on Vercel returns HTML instead of JSON	VITE_API_BASE_URL missing and vercel.json rewrite points elsewhere	Set VITE_API_BASE_URL to the Render URL and redeploy
First request takes a minute	Free instance was asleep	Normal on Free; upgrade to Starter
Uploaded images vanish after a deploy	Cloudinary not configured	Add the three CLOUDINARY_* variables and re-upload
OTP never arrives	Twilio not configured (mock mode) or phone not joined to sandbox	Add Twilio keys; join the sandbox; check Render logs for the OTP in mock mode
Register page says setup already complete	An admin already exists	Log in, or ask a super admin to create your account
Razorpay popup does not open	VITE_RAZORPAY_KEY_ID missing on Vercel	Add it and redeploy
Refresh on /orders shows Vercel 404	SPA rewrite missing	vercel.json in the repo handles it; make sure Root Directory is web / admin
Mobile app: 'Network request failed'	EXPO_PUBLIC_API_URL wrong or phone offline	Check mobile/.env, restart expo start with --clear

14. Final checklist

Tick these before inviting real customers:

- ☐ Render: `NODE_ENV=production`, random `JWT_SECRET`, correct `DATABASE_URL`, `WEB_CLIENT_URL` and `ADMIN_CLIENT_URL` set to the live URLs.
- ☐ Render: Cloudinary keys added; a test product image survives a redeploy.
- ☐ Render: Starter plan (no sleeping) and a paid Postgres plan with backups.
- ☐ Vercel web: `VITE_API_BASE_URL`, `VITE_RAZORPAY_KEY_ID` (live key), `VITE_SUPPORT_WHATSAPP`.
- ☐ Vercel admin: `VITE_API_BASE_URL`; Deployment Protection enabled.
- ☐ First super admin created via `/register`; the page now says setup complete.
- ☐ Test order end to end: add to cart → checkout → COD and Razorpay test payment → order visible in admin → cancel refunds correctly.
- ☐ Twilio WhatsApp sender approved; OTP login tested on a real phone.
- ☐ Google sign-in tested (if enabled).
- ☐ Mobile app built with EAS, pointed at the live API, tested on Android and iOS.
- ☐ Custom domains attached and every stored URL updated.
- ☐ Someone on the team knows how to read Render logs and the Audit Log.

Reference files in the repository

File	Purpose
CEREBRUM.md	Project brain: architecture, flows, test recipes, changelog, debugging cheat-sheet
render.yaml	Render blueprint (database + API + env vars)
web/vercel.json, admin/vercel.json	Vercel config: Vite build, SPA rewrites, /api and /uploads proxy to Render
backend/.env.example, web/.env.example, admin/.env.example	Every environment variable with a comment
docker-compose.yml	Alternative: run everything on one server with Docker